

Формальные языки и грамматики

Теория автоматов и формальных языков

Лекция №8

Структура лекции

- Основные понятия и определения.
- Алфавит, строки, формальный язык, грамматики.
- Форма Бэкуса-Наура.
- Левосторонний и правосторонний, восходящий-нисходящий грамматический разбор.
- Классификация грамматик Хомского.
- Распознавание регулярных грамматик.
- Конечный автомат и его реализация.

Алфавит

- Алфавит – непустое конечное множество символов, используемых для записи предложений языка. Например, множество арабских цифр, знаки «+» и «-»: $A = \{0,1,2,3,4,5,6,7,8,9,+,-\}$ – алфавит для записи целых десятичных чисел, например: «-365», «78», «+45».

Строки алфавита

- Строка – любая последовательность символов алфавита, расположенных один за другим. Строки в теории формальных языков обозначаются строчными греческими буквами: α , β , γ Строка, содержащая 0 символов, называется пустой и обозначается символами ϵ , ε или λ .
- Множество всех составленных из символов алфавита A строк, в которое входит пустая строка, обозначают A^* . Множество всех составленных из символов алфавита A строк, в которое не входит пустая строка, обозначают A^+ . Откуда: $A^* = A^+ \cup \{\epsilon\}$.

Формальный язык

- Формальным языком L в алфавите A называют произвольное подмножество множества A^* . Таким образом, язык определяется как множество допустимых предложений.
- Формальная грамматика – это математическая система, определяющая язык посредством порождающих правил – правил продукции.
- Она определяется как четверка: $G = (V_T, V_N, P, S)$,
где V_T – алфавит языка или множество терминальных (незаменяемых) символов;
 V_N – множество нетерминальных (заменяемых) символов – вспомогательный алфавит, символы которого обозначают допустимые понятия языка, $V_T \cap V_N = \emptyset$;

Грамматика языка

- $V = V_T \cup V_N$ – словарь грамматики;
- P – множество порождающих правил – каждое правило состоит из пары строк (α, β) , где $\alpha \in V^+$ – левая часть правила, $\beta \in V^*$ – правая часть правила: $\alpha \rightarrow \beta$, где строка α должна содержать хотя бы один нетерминал;
- $S \in V_N$ – начальный символ, аксиома грамматики.

Пример записи грамматики

Определим грамматику записи десятичных чисел G_0 :

- $V_T = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -\}$;
- $V_N = \{\langle \text{целое} \rangle, \langle \text{целое без знака} \rangle, \langle \text{цифра} \rangle, \langle \text{знак} \rangle\}$;
- $P = \{\langle \text{целое} \rangle \rightarrow \langle \text{знак} \rangle \langle \text{целое без знака} \rangle,$
 $\langle \text{целое} \rangle \rightarrow \langle \text{целое без знака} \rangle,$
 $\langle \text{целое без знака} \rangle \rightarrow \langle \text{цифра} \rangle \langle \text{целое без знака} \rangle,$
//правосторонняя рекурсия
 $\langle \text{целое без знака} \rangle \rightarrow \langle \text{цифра} \rangle,$
 $\langle \text{цифра} \rangle \rightarrow 0, \langle \text{цифра} \rangle \rightarrow 1, \langle \text{цифра} \rangle \rightarrow 2, \langle \text{цифра} \rangle \rightarrow 3,$
 $\langle \text{цифра} \rangle \rightarrow 4, \langle \text{цифра} \rangle \rightarrow 5, \langle \text{цифра} \rangle \rightarrow 6, \langle \text{цифра} \rangle \rightarrow 7,$
 $\langle \text{цифра} \rangle \rightarrow 8, \langle \text{цифра} \rangle \rightarrow 9,$
 $\langle \text{знак} \rangle \rightarrow +, \langle \text{знак} \rangle \rightarrow - \}$;
- $S = \langle \text{целое} \rangle$.

Форма Бэкуса-Наура (БНФ)

- БНФ связывает терминальные и нетерминальные символы, используя две операции: « $::=$ » – «можно заменить на»; « $|$ » – «или».

$\langle \text{целое} \rangle ::= \langle \text{знак} \rangle \langle \text{целое без знака} \rangle | \langle \text{целое без знака} \rangle,$

$\langle \text{целое без знака} \rangle ::= \langle \text{цифра} \rangle \langle \text{целое без знака} \rangle$
 $| \langle \text{цифра} \rangle (\text{правосторонняя рек.}),$

$\langle \text{цифра} \rangle ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9,$

$\langle \text{знак} \rangle ::= + | - .$

Понятие грамматического разбора

- Исходная строка, строки, получаемые в процессе вывода и аксиома, называются *сентенциальными формами*. *Сентенциальная форма*, содержащая только терминальные символы, называется **допустимой** и представляет собой предложение языка.
- Пример. Вывод строки «-45»:

$\langle \text{целое} \rangle \Rightarrow_1$

$\Rightarrow_1 \langle \text{знак} \rangle \langle \text{целое без знака} \rangle \Rightarrow_2$

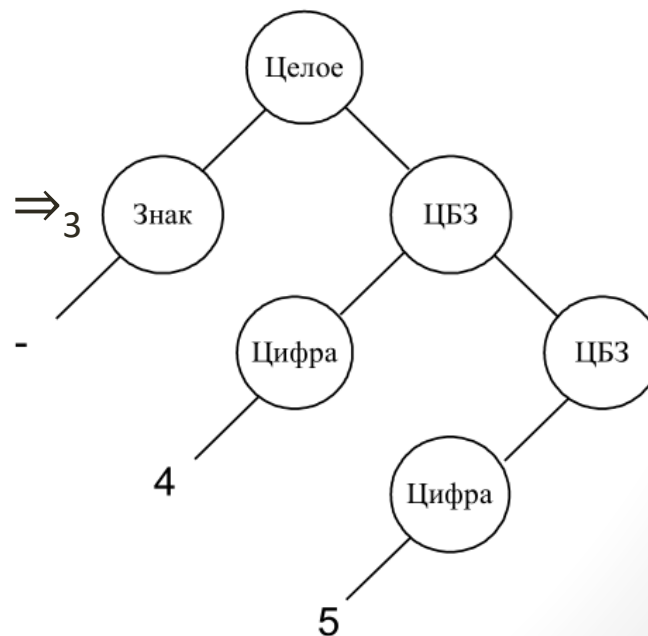
$\Rightarrow_2 \langle \text{знак} \rangle \langle \text{цифра} \rangle \langle \text{целое без знака} \rangle \Rightarrow_3$

$\Rightarrow_3 \langle \text{знак} \rangle \langle \text{цифра} \rangle \langle \text{цифра} \rangle \Rightarrow_4$

$\Rightarrow_4 - \langle \text{цифра} \rangle \langle \text{цифра} \rangle \Rightarrow_5$

$\Rightarrow_5 - 4 \langle \text{цифра} \rangle \Rightarrow_6$

$\Rightarrow_6 - 45$

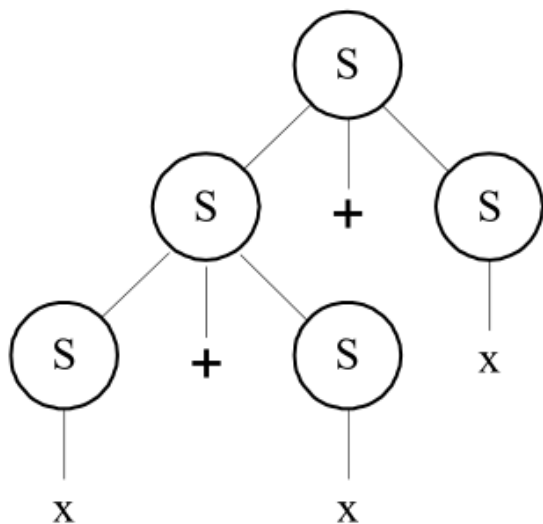


Синтаксическое дерево

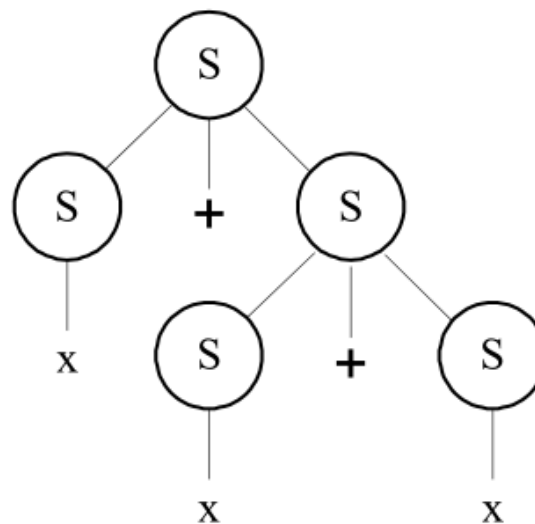
Неоднозначные грамматики

- Грамматика, в которой для одной строки можно построить несколько деревьев называется **неоднозначными**.
- Пример:

Строка $x+x+x$, порождаемая грамматикой с правилами $S \rightarrow S + S$ и $S \rightarrow x$, имеет два дерева разбора



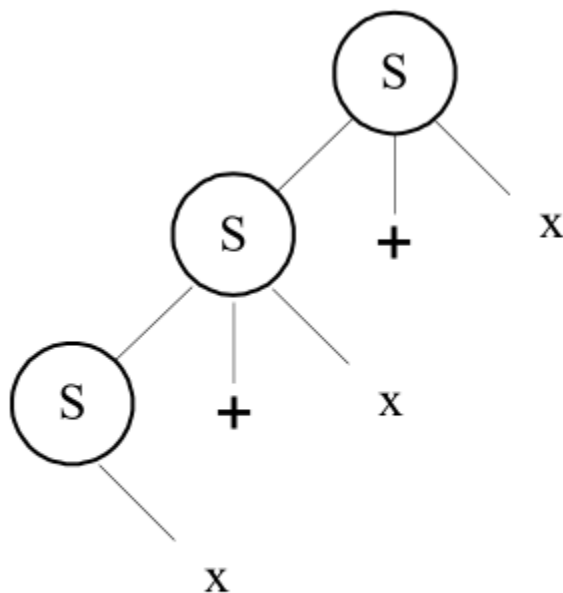
a



b

Неоднозначные грамматики

- Описывающая тот же язык однозначная грамматика использует правила: $S \rightarrow S + x$ и $S \rightarrow x$



Грамматический разбор

- **Грамматический разбор** – процедура построения синтаксического дерева для конкретного предложения языка.
- Построение такого дерева позволяет однозначно доказать, что анализируемая строка языка является допустимой, т.е. принадлежит конкретному языку.
- Существуют нисходящий и восходящий методы разбора. При этом предложения языка можно рассматривать слева направо и наоборот.
- Различают левосторонний и правосторонний разбор.

Левосторонний восходящий грамматический разбор

Разобрать строку «-45»

- Правила грамматики: $\langle \text{целое} \rangle ::= \langle \text{знак} \rangle \langle \text{целое без знака} \rangle | \langle \text{целое без знака} \rangle$,
- $\langle \text{целое без знака} \rangle ::= \langle \text{целое без знака} \rangle \langle \text{цифра} \rangle | \langle \text{цифра} \rangle$,
- $\langle \text{цифра} \rangle ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9$,
- $\langle \text{знак} \rangle ::= + | -$.

Последовательность получения сентенциальных форм данным методом:

- $\langle \text{знак} \rangle 45$
- $\langle \text{знак} \rangle \langle \text{цифра} \rangle 5$
- $\langle \text{знак} \rangle \langle \text{цбз} \rangle 5$
- $\langle \text{целое} \rangle 5$ – Тупик! Возвращаемся и ищем другой вариант подстановки!
- $\langle \text{знак} \rangle \langle \text{целое} \rangle 5$ – Тупик! Возвращаемся и ищем другой вариант подстановки!
- $\langle \text{знак} \rangle \langle \text{цбз} \rangle \langle \text{цифра} \rangle$
- $\langle \text{целое} \rangle \langle \text{цифра} \rangle$ – Тупик! Возвращаемся и ищем другой вариант подстановки!
- $\langle \text{знак} \rangle \langle \text{целое} \rangle \langle \text{цифра} \rangle$ – Тупик! Возвращаемся и ищем другой вариант!
- $\langle \text{знак} \rangle \langle \text{цбз} \rangle$
- $\langle \text{целое} \rangle$

Левосторонний нисходящий грамматический разбор («сверху-вниз»)

- Разобрать строку «-45»
- Правила грамматики:
 - а) $\langle \text{целое} \rangle ::= \langle \text{знак} \rangle \langle \text{целое без знака} \rangle \mid \langle \text{целое без знака} \rangle$
 - б) $\langle \text{целое без знака} \rangle ::= \langle \text{цифра} \rangle \langle \text{целое без знака} \rangle \mid \langle \text{цифра} \rangle,$
 - в) $\langle \text{цифра} \rangle ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9,$
 - г) $\langle \text{знак} \rangle ::= + \mid - \mid .$

Левосторонний нисходящий грамматический разбор («сверху-вниз»)

	Распознано	Текущий символ	Цель или подцель	Правило	Истина?
1		-	Целое	a1	да
2		-	Знак	г1	нет
3				г2	да
4	-	4	Цбз	б1	да
5		4	Цифра	в1-в4	нет
6				в5	да
7	- 4	5	Цбз	б1	нет
8		5	Цифра	в1-в5	нет
9				в6	да
10	-45	—	Цбз	б1	нет
11		—	Цифра	в1-в10	нет
12		—	Цбз	б2	нет
13		—	Цифра	в1-в10	нет
14	- 4	5	Цбз	б2	да
15		5	Цифра	в1-в5	нет
16				в6	да

Расширенная классификация грамматик Хомского

В зависимости от вида правил грамматики различают:

тип 0 – грамматики фразовой структуры или грамматики «без ограничений»:

$\alpha \rightarrow \beta$, где $\alpha \in V^+$, $\beta \in V^*$ – в таких грамматиках допустимо наличие любых правил вывода, что свойственно грамматикам естественных языков;

тип 1 – контекстно-зависимые (неукорачивающие) грамматики:

$\alpha \rightarrow \beta$, где $\alpha \in V^+$, $\beta \in V^*$, $|\alpha| \leq |\beta|$ – в этих грамматиках для правил вида $\alpha X \beta \rightarrow \alpha x \beta$ возможность подстановки строки x вместо символа X определяется присутствием подстрок α и β , т. е. контекста, что также свойственно грамматикам естественных языков;

(расширение допускает не более одного правила вида $A \rightarrow e$, где $A \in V_N$);

тип 2 – контекстно-свободные грамматики:

$A \rightarrow \beta$, где $A \in V_N$, $\beta \in V^*$ – поскольку в левой части правила стоит нетерминал, подстановки не зависят от контекста;

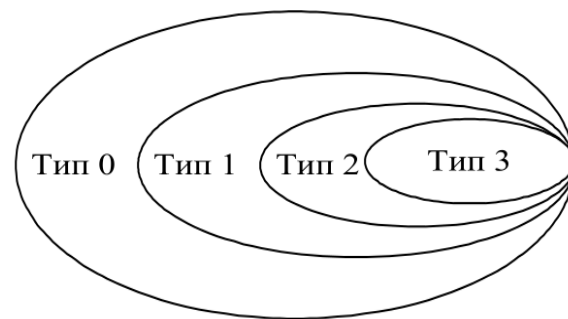
тип 3 – регулярная грамматика:

$A \rightarrow \alpha$, $A \rightarrow \alpha B$, где $A, B \in V_N$, $\alpha \in V_T$;

(расширение допускает не более одного правила вида $S \rightarrow e$, но в этом случае аксиома не должна появляться в правых частях правил).

Расширенная классификация грамматик Хомского

- Классификация построена по правилам иерархии, т. е. грамматики типа 3 являются частным случаем грамматик типа 2 и т. п.
- Грамматики большинства существующих языков программирования относятся к типу 2



Вложение типов

Простые методы проверки регулярности языка (тип 3)

- Существует простой метод проверки регулярности языка, основанный на *лемме о разрастании*:

в строке регулярного языка всегда можно найти непустую подстроку, повторение которой произвольное количество раз порождает новые строки того же языка.

- язык не относится к классу регулярных, если присутствует *самовложение*, т. е. наличие правил вида:

$A \Rightarrow + \alpha_1 A \alpha_2$, где $\alpha_1, \alpha_2 \in V^+$.

Распознавание типа языка

Проверка соответствия строк языку осуществляется специальной программой – **распознавателем**. Распознаватели могут использоваться для определения языка также как и грамматики. Чем шире класс распознаваемых грамматик, тем сложнее класс соответствующих распознавателей. Доказано, что:

- грамматики типа 3 распознаются конечными автоматами;
- грамматики типа 2 распознаются автоматами с магазинной памятью;
- грамматики типа 1 распознаются линейными ограниченными автоматами;
- грамматики типа 0 распознаются машинами Тьюринга.

Распознавание регулярных грамматик

Конечный автомат и его
программная реализация

Распознавание регулярных грамматик

- Распознаватели регулярных грамматик строятся на конечных автоматах.
- **Конечный автомат** – это математическая модель, свойства и поведение которой полностью определяются пятеркой:
 $M = (Q, \Sigma, \delta, q_0, F),$

где Q – конечное **множество состояний**;

Σ – конечное **множество входных символов**;

$\delta(q_i, c_k)$ – **функция переходов** (q_i – текущее состояние, c_k – очередной символ);

q_0 – **начальное состояние**;

$F = \{q_j\}$ – подмножество **допускающих состояний**.

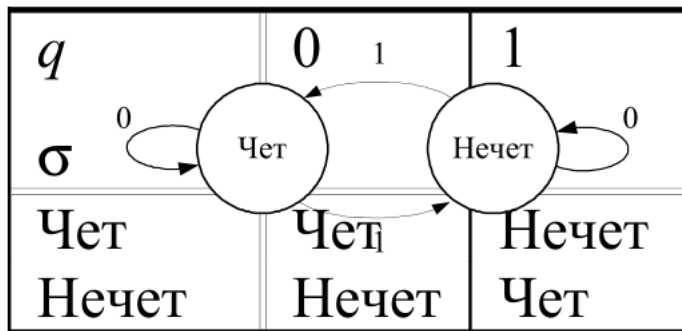
Пример конечного автомата

Пример. Автомат «**Чет-нечет**» описывается следующим образом:

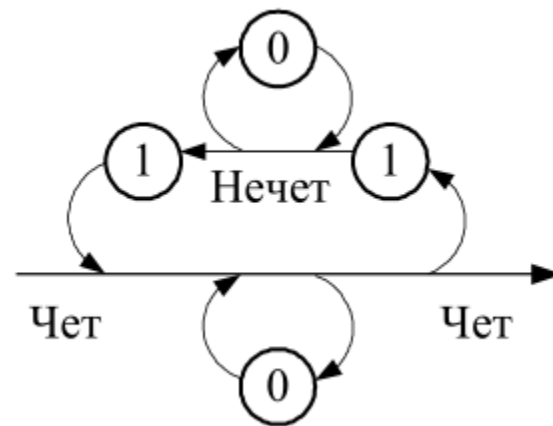
- $Q = \{\text{Чет}, \text{Нечет}\};$
- $\Sigma = \{0, 1\};$
- $\delta(\text{Чет}, 0) = \text{Чет},$
 $\delta(\text{Нечет}, 0) = \text{Нечет},$
 $\delta(\text{Чет}, 1) = \text{Нечет},$
 $\delta(\text{Нечет}, 1) = \text{Чет};$
- $q_0 = \text{Чет};$
- $F = \{\text{Чет}\}$

Строка «10110» приведет такой автомат в состояние **Нечет** и будет отвергнута, а строка «110011» – в состояние **Чет** и будет принята.

Представление функции переходов



Граф переходов состояний



Синтаксическая диаграмма

q	c	0	1	Символы завершения	Другие символы
Чет	Чет	Чет	Не-чет	Конец	Ошибка
Нечет	Не-чет	Не-чет	Чет	Ошибка	Ошибка

Таблица состояний

Алгоритм работы автомата

Пусть **S** – строка на входе автомата;

Ind – номер очередного символа;

q – текущее состояние автомата;

Table – таблица, учитывающая символы завершения и другие символы.

$Ind := 1$

$q := q_0$

Цикл-пока $q \neq \text{«Ошибка»}$ и $q \neq \text{«Конец»}$

$q = Table [q, S[Ind]]$

$Ind := Ind + 1$

Все-цикл

Если $q = \text{«Конец»}$

то «Строка принята»

иначе «Строка отвергнута»

Все-если

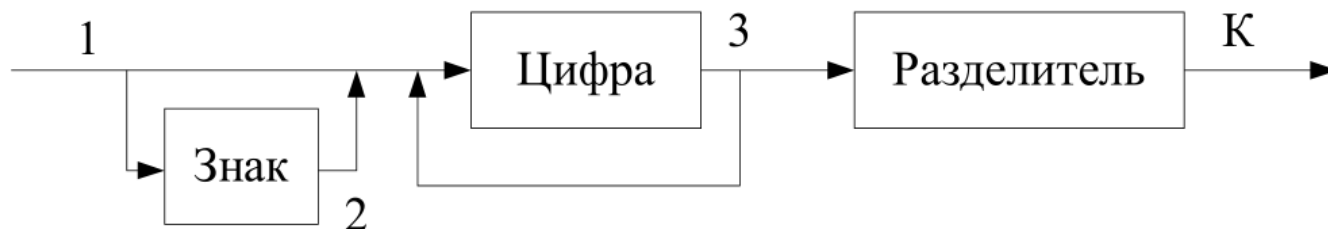
Обобщенный алгоритм работы автомата на языке C

```
char c;           //текущий исходный символ
int q;            //номер состояния
int a;            //входной текущий символ для автомата
q=0;              //начальное состояние автомата
while(1)          //бесконечный цикл
{
    c = readchar(); //считывание входного символа
    a = gettype(c);  //распознавание входного символа –
                     //отнесение его к одной из известных автомату
                     //категорий - <бкв>, <цфр>, <конец> или <иначе>

    //Выполнение перехода
    q = P[a, q];
    //Обработка
    if (q==3) return 1; //нормальный выход из программы
    if (q==4) return 0; //выход по ошибке
}
```

Построение лексических анализаторов

- Конечные автоматы используют для распознавания лексем второго типа (базовых понятий языка). Эти понятия можно описать в рамках самых простых, регулярных грамматик и предложить для них эффективную технику разбора.
- Пример. Распознаватель целых чисел. Синтаксическая диаграмма:



1 – начало разбора; 2 – распознан знак; 3 – целое; К – завершение разбора

Построение таблицы переходов

- По диаграмме строим таблицу переходов (см. таблицу 5), обозначая состояние ошибки символом «Е». В таблице переходов указываем подпрограммы обработки, которые должны быть выполнены при осуществлении указанного перехода. При выполнении этих подпрограмм формируются указанные значения.

	Знак	Цифра	Разделитель	Другие
1	2, А1	3, А2	Е, Д1	Е, Д4
2	Е, Д2	3, А2	Е, Д3	Е, Д4
3	К, А3	3, А2	К, А3	Е, Д4

а) Подпрограммы обработки:

А0: Инициализация: Целое := 0;

Знак_числа := «+».

А1: Знак_числа := Знак

А2: Целое := Целое*10 + Цифра

А3: Если Знак_числа = «-» то Целое :=
-Целое Все-если

б) Диагностические сообщения:

Д1: «Строка не является десятичным
числом»;

Д2: «Два знака рядом»;

Д3: «В строке отсутствуют цифры»,

Д4: «В строке встречаются
недопустимые символы»

Программная реализация

- Обозначим: **S** – строка на входе автомата; **Ind** – номер очередного символа; **q** – текущее состояние автомата; **Table** – таблица, учитывающая символы завершения и другие символы.
- $\text{Ind} := 1$
- $q := 1$
- Выполнить A_0

- $Ind := 1$
- $q := 1$
- Выполнить A_0
- Цикл-пока $q \neq \text{«E»}$ и $q \neq \text{«K»}$
 - Если $S[Ind] = \text{«+»}$ или $S[Ind] = \text{«-»}$,
 - то $j := 1$
 - иначе Если $S[Ind] \geq \text{«0»}$ и $S[Ind] \leq \text{«9»}$,
 - то $j := 2$,
 - иначе $j := 3$
 - Все-если
 - Все-если
 - Выполнить $A_i := \text{Table}[q, j].A()$
 - $q := \text{Table}[q, j]$
 - $Ind := Ind + 1$
- Все-цикл
- Если $q = \text{«K»}$
 - то Выполнить A_3
 - Вывести сообщение «Это число»
 - иначе Вывести сообщение D_i
- Все-если

Построение синтаксических анализаторов

- **Пример.** Синтаксический анализатор списка описания целых скаляров, массивов и функций (упрощенный вариант)
- После лексического анализа входная строка представлена в алфавите:
- **V** – идентификатор; **N** – целочисленная константа; служебные символы: «[] () , ; *».
- Функцию переходов зададим синтаксической диаграммой:

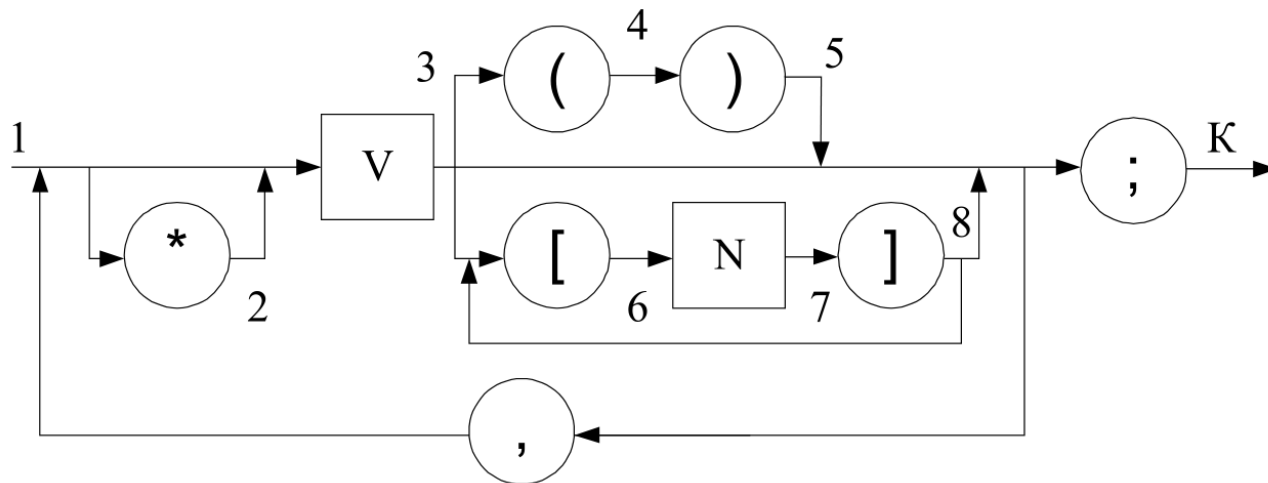


Таблица переходов

	V	N	*	(	)	[	]	,	;	Другие
1	3	E	2	E	E	E	E	E	E	E
2	3	E	E	E	E	E	E	E	E	E
3	E	E	E	4	E	6	E	1	K	E
4	E	E	E	E	5	E	E	E	E	E
5	E	E	E	E	E	E	E	1	K	E
6	E	7	E	E	E	E	E	E	E	E
7	E	E	E	E	E	E	8	E	E	E
8	E	E	E	E	E	6	E	1	K	E

Алгоритм распознавателя

Ind := 1

q := 1

Цикл-пока $q \neq \text{«E»}$ и $q \neq \text{«K»}$

$q := \text{Table}[q, \text{Pos}(S[\text{Ind}], \text{«VN*()[];»})]$

 Ind := Ind + 1

Все-цикл

Если $q = \text{«K»}$

 то Выполнить A3

 Вывести сообщение «Это число»

 иначе Вывести сообщение D_i

Все-если

Контрольные вопросы

1. Определите, что такое алфавит языка.
2. Дайте определение формального языка. Почему формальные языки обычно не определяют перечислением допустимых предложений?
3. Дайте определение формальной грамматики.
4. Что такое «Форма Бэкуса-Наура»? Для чего она используется?
5. Определите цель грамматического разбора предложений языка.
6. В чем заключается левосторонний восходящий грамматический разбор? Назовите, в каких случаях он не применим и определите его основной недостаток.
7. Назовите основные типы грамматик по Хомскому. Какие грамматики используют в языках программирования?
8. Регулярные выражения и конечные автоматы.
9. Детерминированные конечные автоматы.
10. Синтаксические деревья.

Задание 1.

Бескомпьютерная обработка

- Сформировать формальный язык, позволяющий генерировать предложения и словосочетания из указанных элементов:
 - Существительные (не менее пяти);
 - Прилагательные (не менее трех);
 - Глаголы (не менее пяти);
 - Предлоги (не менее трех).
- Выработать правила объединения слов:
 - (существительное+прилагательное, существительное+глагол, прилагательное+существительное+глагол, существительное+предлог+глагол и т.д.);
 - желательно использовать такие нетерминалы, как: словосочетание, подлежащее, сказуемое, дополнение и т.д.
 - цепочки могут иметь рекурсии.
- Написать примеры сформированных предложений (не менее 5):
 - раскрыть цепочки формирующие предложения, объяснить проблемы в употреблении некоторых сочетаний.

Задание 2.

Компьютерная обработка

- Для предложенной КС-грамматики написать программу, формирующую предложения на основе таблиц слов:
 - указывать последовательность получения предложений из аксиомы грамматики;
 - выводить полученные предложения;
 - исходные слова можно задавать непосредственно в коде программы, либо в подключаемом файле (не нужно делать ввод с клавиатуры в процессе выполнения).
- В отчете оформить таблицы символов, лексем, имен, описать КС-грамматику и выводимые программой предложения.

Задание 3.

Дополнительное

- Предложить возможные варианты улучшения КС-грамматики для более корректной передачи особенностей языка (падежи, род, множественное число и др.)
- В соответствии с предложенными идеями модифицировать формальный язык, добавив к словам указанные параметры.
 - Требуется также модифицировать правила вывода грамматики для получения «хороших» предложений.
- Предложить способы хранения таблиц лексем, имен, идентификаторов в программе с использованием конструкций языка программирования.